

Student Name:

Student ID:

Habib University
Dhanani School of Science and Engineering
Computer Architecture
EE 371 / CS 330 / CE 321 (Fall 2024)

Quiz 2 1

Date: 7th Oct., 2024

Total marks: 20

Obtained Marks:

Total time: 50 mins

Instructions:

1. All questions should be answered **with ink pen**.
2. Smart watches, laptops, and similar electronics are **strictly NOT allowed**.
3. Answer sheets should contain all steps, working, explanations, and assumptions.
4. Write your name and HU ID on all sheets.
5. Turn in your question paper along with your answer sheets.
6. You are not allowed to ask/share your method or answer with your peers. The work submitted by you is solely your own work. Any violation of this will be the violation of HU Honor code and proper action will be taken as per university policy if found to be involved in such an activity.

CLO Assessment:

This quiz assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	✓
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	
CLO 4	<i>Analyze the</i> performance of cache operations	

Student Name:

Student ID:

Q1: Write assembly language instructions for the give C code. Use same registers as variable names. Use the RISC-V procedure call convention. Remember that *x5*, *x6* and *x7* are temporary registers and do not need stack. However, return address for calling a function (*jal* & *jalr* instruction) uses *x1* register, which needs stack. (6 points)

```
main(){
    x5 = x5 + 5;
    funct_1();
    return;
}
```

```
funct_1(){
    x6 = x6 + 6;
    funct_2();
    return;
}
```

```
funct_2(){
    x7 = x7 + 7;
    return;
}
```

Sol:

```
main:      addi    x5, x5, -5
           jal     x1, funct_1

funct_1:   addi    sp, sp, -8
           sd      x1, 0(sp)          # since funct_1 is calling funct_2, we need to store x1
           addi    x6, x6, -6
           jal     x1, funct_2        # x1 over-written here
           ld      x1, 0(sp)
           addi    sp, sp, 8
           jalr    x0, 0(x1)

funct_2:   addi    x7, x7, -7
           jalr    x0, 0(x1)
```

Student Name:

Student ID:

Q2: Multiply 1010_2 with 1101_2 using multiplication algorithm. Assume that instead of 0, we have a non-zero value in product at start (7 points)

Sol:

Iteration	Step	Multiplier(a)	Multiplicand(b)	Product
0		1010 (10)	0000 1101 (13)	1100 0011 (195)
1	0 → No operation Shift right a Shift left b	1010 0101 0101	0000 1101 0000 1101 0001 1010	1100 0011 1100 0011 1100 0011
2	1 → Prod + b Shift right a Shift left b	0101 0010 0010	0001 1010 0001 1010 0011 0100	1101 1101 1101 1101 1101 1101
3	0 → No operation Shift right a Shift left b	0010 0001 0001	0011 0100 0011 0100 0110 1000	1101 1101 1101 1101 1101 1101
4	1 → Prod + b Shift right a Shift left b	0001 0000 0000	0110 1000 1101 0000 1101 0000	1 0100 0101 1 0100 0101 1 0100 0101

$$13 * 10 + 195 = 325$$

Student Name:

Student ID:

Q3: Convert 3125.03125 to IEEE 754 single precision floating point number (7 points)

Sol:

3125 $\rightarrow$ 110000110101

0.03125 $\rightarrow$ 0.00001

3125.03125 = 110000110101.00001

Normalized scientific form: $1.1000011010100001 * 2^{11}$

Exp = $127 + 11 = 138 = 10001010$

Sign = 0

Fraction/mantissa = 1000011010100001

Representation = 0 10001010 100001101010000100000000

Student Name:

Student ID:

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	①110 0111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	①111 0111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	①111 1111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	①000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ SLL Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	①000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ SLL Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001①	0000 0010	0000 0000
1	1a: 1 $\Rightarrow$ Prod = Prod + Mcand	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000①	0000 0100	0000 0010
2	1a: 1 $\Rightarrow$ Prod = Prod + Mcand	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000①	0000 1000	0000 0110
3	1: 0 $\Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000①	0001 0000	0000 0110
4	1: 0 $\Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

Stack above the stack pointer

Stack below the stack pointer